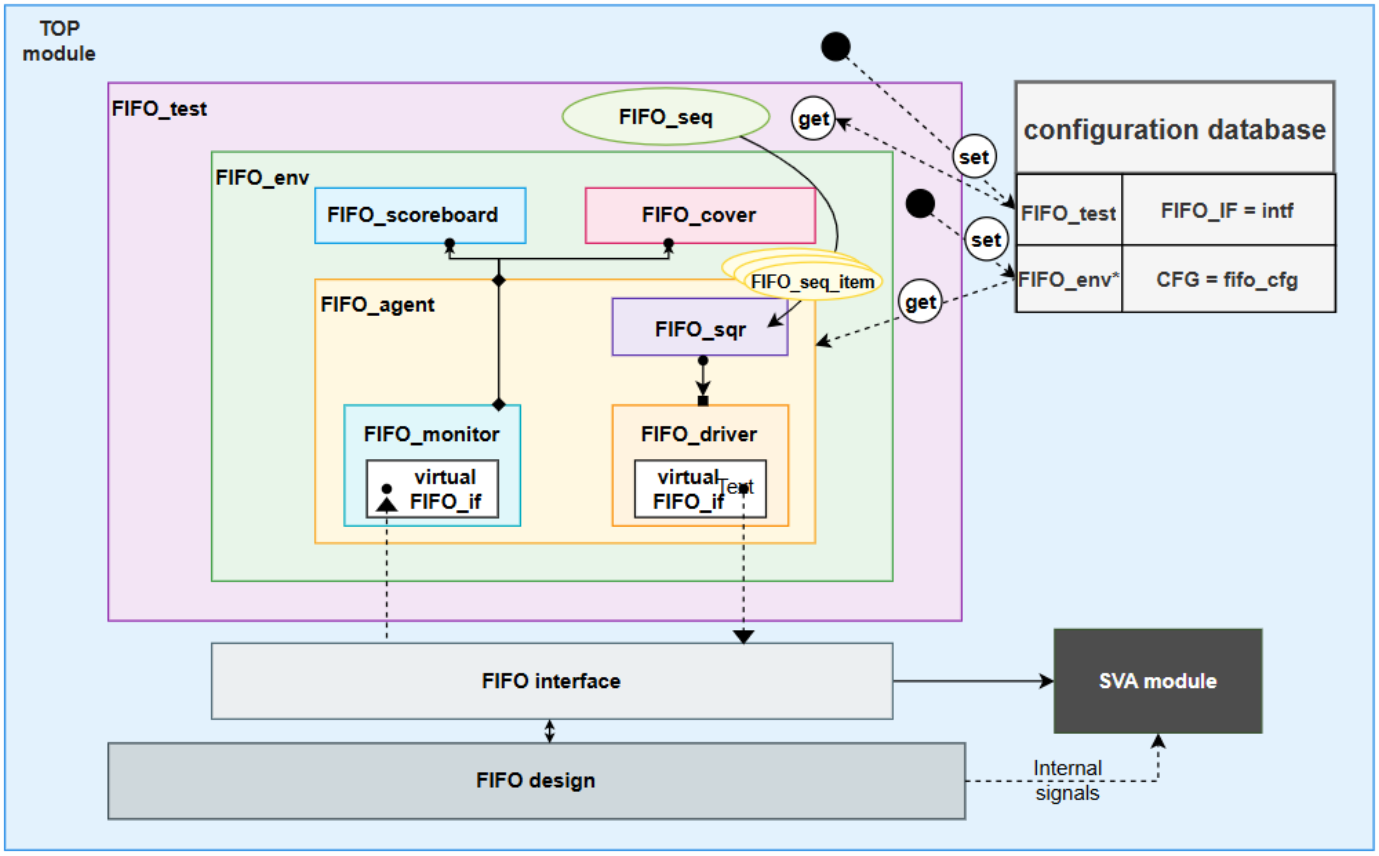
UVM final project solution

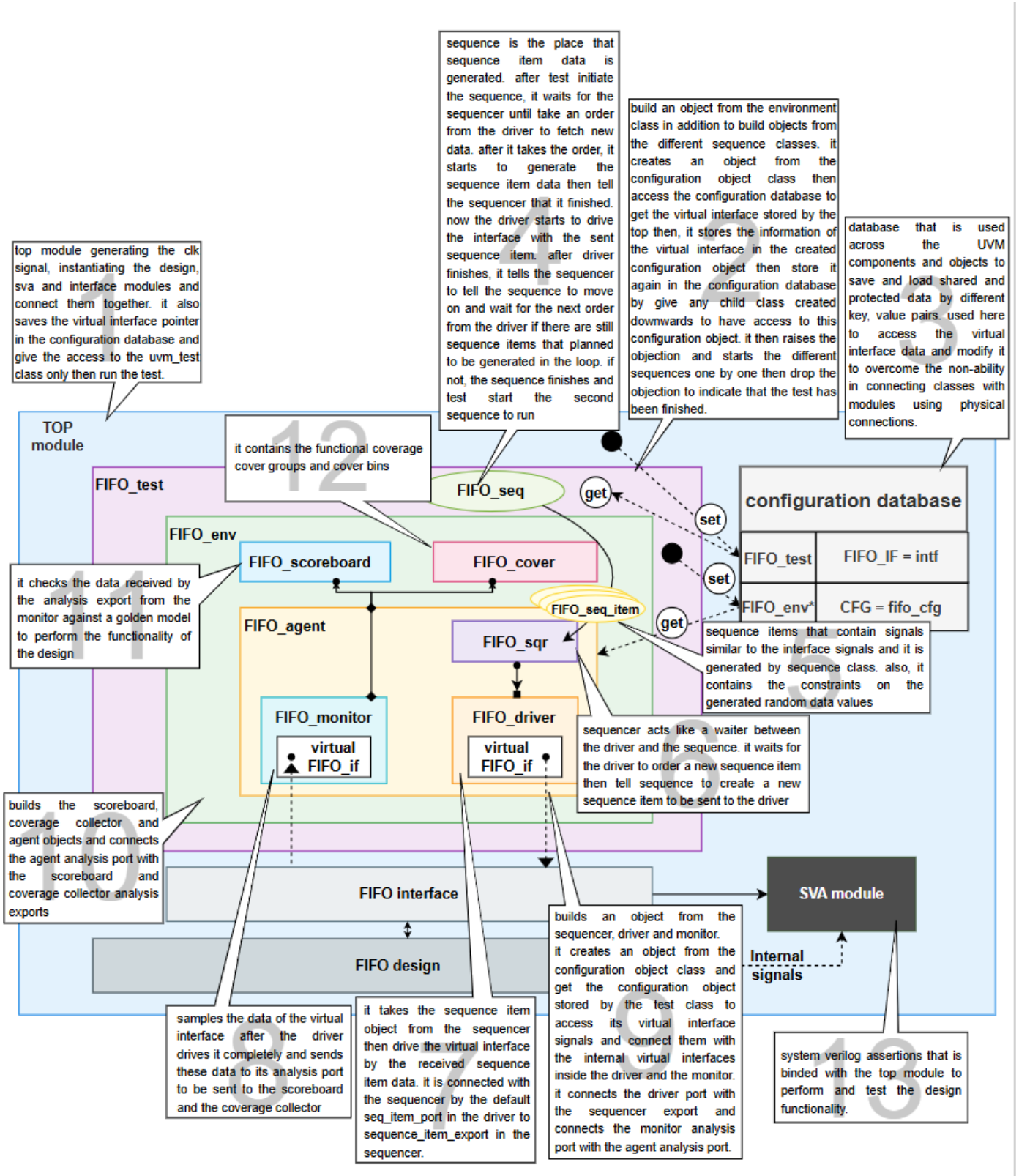
1. Verification plan

	A	B	C	D	E
1	**Label**	**Design Requirement Description**	**Stimulus Generation**	**Functional Coverage**	**Functionality Check**
2	**FIFO_1**	when reset is low the output signals must be low except the empty flag must be raised	direct in the start of the test	-	immediate assertion in the sva module
3	**FIFO_2**	when reset is low internal pointers and counter must be reset to zero	direct in the start of the test	-	immediate assertion in the sva module
4	**FIFO_3**	check that internal pointers and conter not exceed the FIFO_DEPTH value	constrained randomization	-	immediate assertion in the sva module
5	**FIFO_4**	check that data_out is reading properly when read enable raised while fifo is not full yet	constrained randomization	-	compare with a golden model
6	**FIFO_5**	check that data_out is not reading when the read enable raised but the fifo is empty	constrained randomization	-	compare with a golden model
7	**FIFO_6**	check that data_out is not reading when the read and write enable are raised simultaneuosly while fifo is empty and only the write operation is done	constrained randomization	-	compare with a golden model
8	**FIFO_7**	check that only read operation is done when the read and write enable are raised simultaneuosly while fifo is full	constrained randomization	-	compare with a golden model
9	**FIFO_8**	check that raising of the wr_ack signal when a valid write operation is done	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
10	**FIFO_9**	check raising of the full signal when the fifo become full	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
11	**FIFO_10**	check raising of the empty signal when the fifo become empty	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
12	**FIFO_11**	check raising of the almost full signal when the fifo still have one empty location only	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
13	**FIFO_12**	check raising of the almost empty signal when the fifo still have one data location only	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
14	**FIFO_13**	check raising of the overflow signal when a write operation considered to be done while the fifo is full	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
15	**FIFO_14**	check raising of the underflow signal when a read operation considered to be done while the fifo is empty	constrained randomization	cross coverage with input write and read enable signals	concurrent assertion in the sva module
16	**FIFO_15**	check for pointer wrapping	constrained randomization	-	concurrent assertion in the sva module
17	randomization constraints: reset is 2% on, read enable is 30% on, write enable is 70% on				

2. UVM testbench structure



3. In detail discerption about how the testbench works



5. Bug report (same as sv final project)

- **Output data**

  - ❖ **observation**

    When the FIFO is full and the input write enable and read enable signals are both high the output becomes different from the expected output.

  - ❖ **Cause**

    This scenario is not considered when updating the FIFO's internal counter. When both write and read enables are high, the counter value remains unchanged, rather than decrementing by one—even though only a read operation should be effectively counted in that case. Similarly, when the FIFO is empty and both read and write enables are high, the counter value remains zero instead of incrementing by one, even though only a write operation should be considered in this situation.

```
35  always @(posedge intf.clk or negedge intf.rst_n) begin
36    if (!intf.rst_n) begin
37      count <= 0;
38    end else begin
39      if (({intf.wr_en, intf.rd_en} == 2'b10) && !intf.full) count <= count + 1;
40      else if (({intf.wr_en, intf.rd_en} == 2'b01) && !intf.empty) count <= count - 1;
41    end
42  end
```

  - ❖ **FIX**

```
49  always @(posedge intf.clk or negedge intf.rst_n) begin
50    if (!intf.rst_n) begin
51      count <= 0;
52    end else begin
53      if (({intf.wr_en, intf.rd_en} == 2'b10) && !intf.full) count <= count + 1;
54      else if (({intf.wr_en, intf.rd_en} == 2'b01) && !intf.empty) count <= count - 1;
55      else if (({intf.wr_en, intf.rd_en} == 2'b11) && intf.empty) count <= count + 1;
56      else if (({intf.wr_en, intf.rd_en} == 2'b11) && intf.full) count <= count - 1;
57    end
58  end
```

- **Under flow signal**

  - ❖ **Observation**

    Under flow signal is driven computationally not sequentially

  - ❖ **Error code**

```
43
44  assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
45  assign intf.empty = (count == 0) ? 1 : 0;
    assign intf.underflow = (intf.empty && intf.rd_en) ? 1 : 0;
47  assign intf.almostfull = (count == FIFO_DEPTH - 2) ? 1 : 0;
48  assign intf.almostempty = (count == 1) ? 1 : 0;
49
```

❖ **FIX**

```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    rd_ptr <= 0;
    intf.data_out <= 0;
    intf.underflow <= 0;
  end else if (intf.rd_en && count != 0) begin
    intf.data_out <= mem[rd_ptr];
    rd_ptr <= rd_ptr + 1;
  end else begin
      if(intf.rd_en && intf.empty) intf.underflow <= 1;
      else intf.underflow <= 0;
  end
end
```

- **Almost full signal**

❖ **Observation**

It was found that the almost full signal is asserted one entry earlier than expected. When the FIFO still has two available slots, the signal is raised instead of waiting until only one slot remains before becoming full.

❖ **Error in the code**

```
assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
assign intf.empty = (count == 0) ? 1 : 0;
assign intf.underflow = (intf.empty && intf.rd_en) ? 1 : 0;
assign intf.almostfull = (count == FIFO_DEPTH - 2) ? 1 : 0;
assign intf.almostempty = (count == 1) ? 1 : 0;
```

❖ **FIX**

```
assign intf.full = (count == FIFO_DEPTH) ? 1 : 0;
assign intf.empty = (count == 0) ? 1 : 0;
assign intf.almostfull = (count == FIFO_DEPTH - 1) ? 1 : 0;
assign intf.almostempty = (count == 1) ? 1 : 0;
```

- **Top resetting**

❖ **Observation**

The registered outputs are not reset when the reset signal is asserted, it remains unchanged.

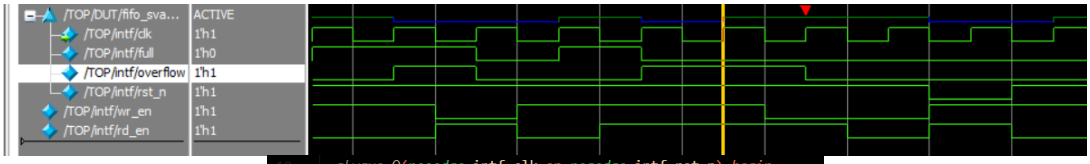
❖ **Fix**

Reset output registered signals which are data_out, wr_ack, overflow, underflow.

6. Extra detected bugs according to adding more assertions test cases (not included in the previous sv final project)

- **Overflow and underflow signals (observation)**

  Look at the scenario in the following waveform and code snippet




```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    wr_ptr <= 0;
  intf.overflow <= 0;
  intf.wr_ack <= 0;
  end else if (intf.wr_en && count < FIFO_DEPTH) begin
    mem[wr_ptr] <= intf.data_in;
    intf.wr_ack <= 1;
    wr_ptr <= wr_ptr + 1;
  end else begin
    intf.wr_ack <= 0;
    if (intf.full & intf.wr_en) intf.overflow <= 1;
    else intf.overflow <= 0;
  end
end
```

  In this scenario, when the FIFO is full and wr_en = 1, the overflow signal is asserted. However, if wr_en remains high and rd_en is asserted, a read operation occurs while the write operation is ignored, causing the FIFO to no longer be full. Therefore, the overflow signal should be deasserted, but instead it remains asserted until wr_en goes low.

- **Fix**

  Add intf.overflow<=0 in the second branch to make sure that it will be zero at any valid write operation.

```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    wr_ptr <= 0;
  intf.overflow <= 0;
  intf.wr_ack <= 0;
  end else if (intf.wr_en && count < FIFO_DEPTH) begin
    mem[wr_ptr] <= intf.data_in;
    intf.wr_ack <= 1;
    wr_ptr <= wr_ptr + 1;
    intf.overflow <= 0;
  end else begin
    intf.wr_ack <= 0;
    if (intf.full & intf.wr_en) intf.overflow <= 1;
    else intf.overflow <= 0;
  end
end
```

  The same issue happened in the underflow signal, and we fixed it with the same method.

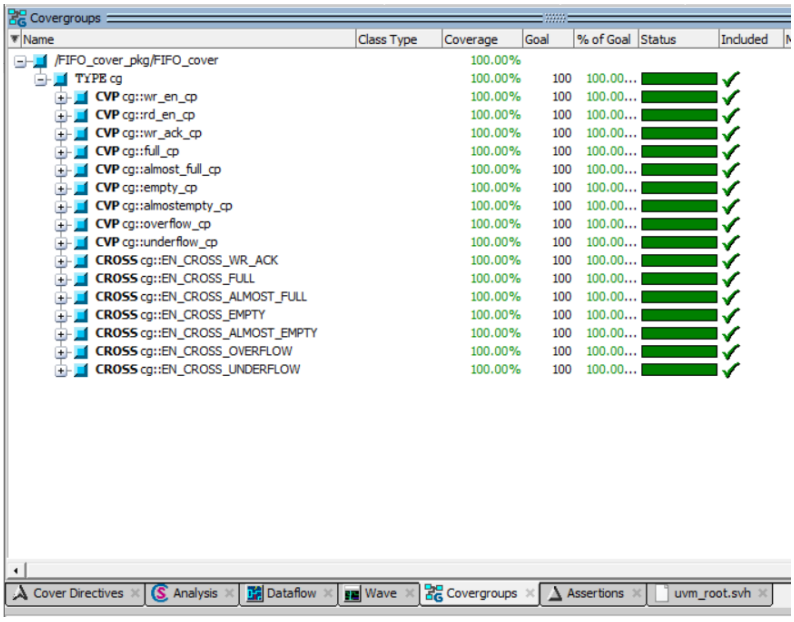
```
always @(posedge intf.clk or negedge intf.rst_n) begin
  if (!intf.rst_n) begin
    rd_ptr <= 0;
    intf.data_out <= 0;
    intf.underflow <= 0;
  end else if (intf.rd_en && count != 0) begin
    intf.data_out <= mem[rd_ptr];
    rd_ptr <= rd_ptr + 1;
    intf.underflow <= 0;
  end else begin
    if(intf.rd_en && intf.empty) intf.underflow <= 1;
    else intf.underflow <= 0;
end
end
```

7. Code coverage report

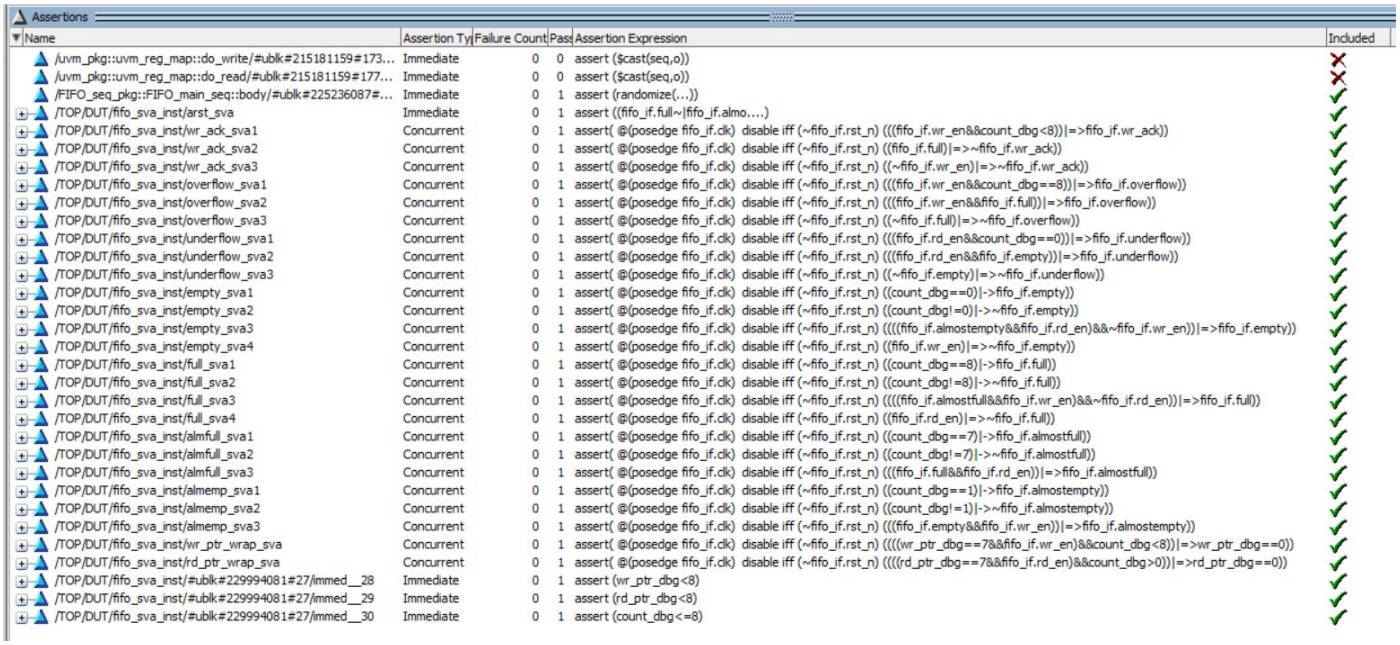
report.txt

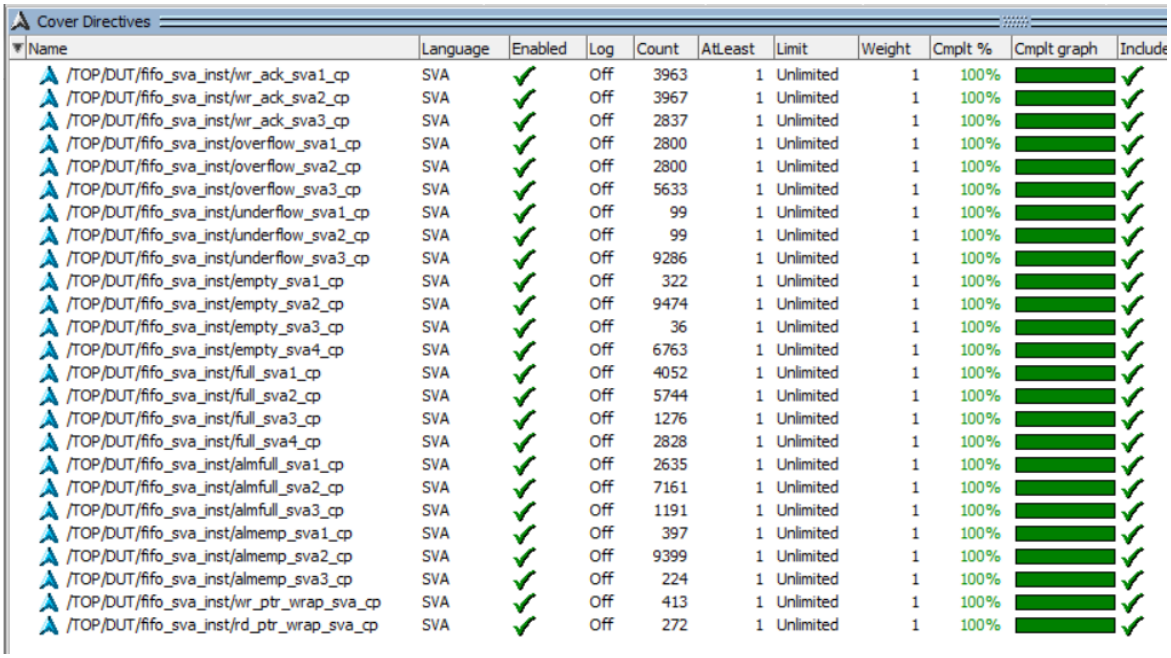
```
Coverage Report Summary Data by instance

================================================================================
=== Instance: /TOP/DUT
=== Design Unit: work.FIFO
================================================================================
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        25        25         0   100.00%
    Statements                      28        28         0   100.00%
    Toggles                         20        20         0   100.00%


Total Coverage By Instance (filtered view): 100.00%
```

8. Functional coverage



9. Assertion results and coverage

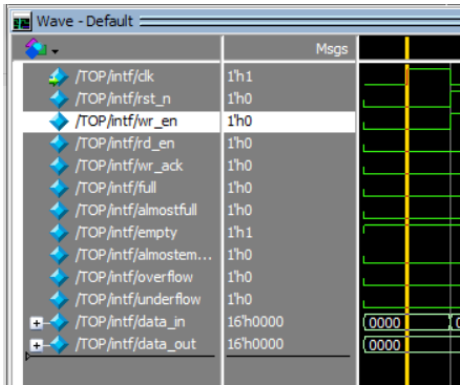




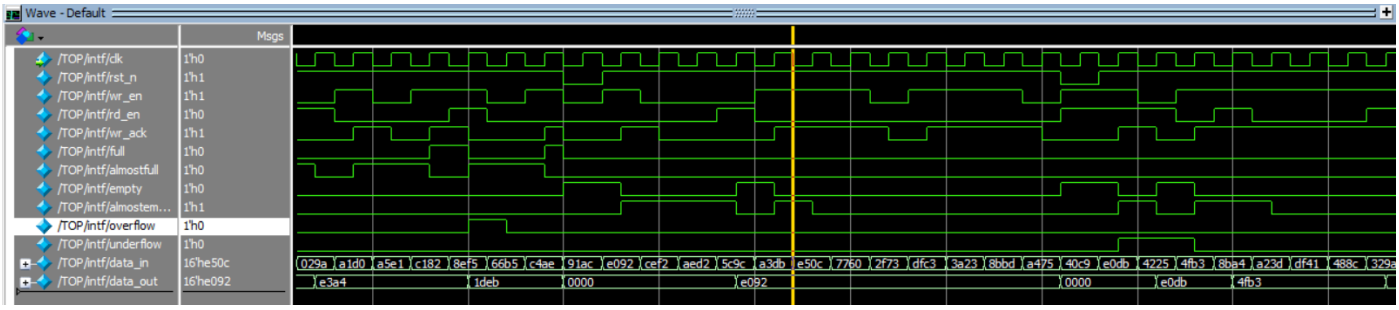
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Include
/TOP/DUT/fifo_sva_inst/wr_ack_sva1_cp	SVA	✓	Off	3963	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/wr_ack_sva2_cp	SVA	✓	Off	3967	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/wr_ack_sva3_cp	SVA	✓	Off	2837	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/overflow_sva1_cp	SVA	✓	Off	2800	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/overflow_sva2_cp	SVA	✓	Off	2800	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/overflow_sva3_cp	SVA	✓	Off	5633	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/underflow_sva1_cp	SVA	✓	Off	99	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/underflow_sva2_cp	SVA	✓	Off	99	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/underflow_sva3_cp	SVA	✓	Off	9286	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/empty_sva1_cp	SVA	✓	Off	322	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/empty_sva2_cp	SVA	✓	Off	9474	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/empty_sva3_cp	SVA	✓	Off	36	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/empty_sva4_cp	SVA	✓	Off	6763	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/full_sva1_cp	SVA	✓	Off	4052	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/full_sva2_cp	SVA	✓	Off	5744	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/full_sva3_cp	SVA	✓	Off	1276	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/full_sva4_cp	SVA	✓	Off	2828	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/almfull_sva1_cp	SVA	✓	Off	2635	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/almfull_sva2_cp	SVA	✓	Off	7161	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/almfull_sva3_cp	SVA	✓	Off	1191	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/almemp_sva1_cp	SVA	✓	Off	397	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/almemp_sva2_cp	SVA	✓	Off	9399	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/almemp_sva3_cp	SVA	✓	Off	224	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/wr_ptr_wrap_sva_cp	SVA	✓	Off	413	1	Unlimited	1	100%		✓
/TOP/DUT/fifo_sva_inst/rd_ptr_wrap_sva_cp	SVA	✓	Off	272	1	Unlimited	1	100%		✓

10. Sequences in waveform

- **Reset sequence**



- **Main sequence**



11. Assertion table

Feature	Assertion	type
When reset is asserted, all registered outputs are reset to zero asynchronously	Direct assertion	■
Whenever wr_en is asserted and internal counter value is less than FIFO_DEPTH means that FIFO is not full, wr_ack is = 1	**@ (posedge fifo_if.clk)** fifo_if.wr_en **&&** count_dbg < FIFO_DEPTH **|=>** fifo_if.wr_ack;	□
Whenever FIFO is full, wr_ack is always = 0	**@ (posedge fifo_if.clk)** fifo_if.full **|=>** **!**fifo_if.wr_ack;	■
Whenever wr_en is 0 the wr_ack is also 0	**@(posedge fifo_if.clk)** **!**fifo_if.wr_en **|=>** !fifo_if.wr_ack;	■

Whenever wr_en is asserted and the internal counter is = FIFO_DEPTH means that FIFO is full, overflow is = 1	**@(posedge fifo_if.clk)** fifo_if.wr_en **&&** count_dbg == FIFO_DEPTH **|=>** fifo_if.overflow;	☐
Whenever wr_en is asserted while FIFO is full, overflow is = 1	**@(posedge fifo_if.clk)** fifo_if.wr_en **&&** fifo_if.full **|=>** fifo_if.overflow;	■
Whenever FIFO is not full, overflow is = 0	**@(posedge fifo_if.clk)** **!**fifo_if.full **|=>** **!**fifo_if.overflow;	■
Whenever internal counter is = 0, the empty signal is = 1	**@(posedge fifo_if.clk)** count_dbg == 0 **|->** fifo_if.empty;	☐
Whenever internal counter is not = 0, the empty signal is = 0.	**@(posedge fifo_if.clk)** count_dbg **!=** 0 **|->** **!**fifo_if.empty;	☐
When the almostempty is asserted and a read operation is required without a write operation then empty signal will be = 1	**@(posedge fifo_if.clk)** fifo_if.almostempty **&&** fifo_if.rd_en **&&** **!**fifo_if.wr_en **|=>** fifo_if.empty;	■
Whenever wr_en is asserted, empty signal must = 0	**@(posedge fifo_if.clk)** fifo_if.wr_en **|=>** **!**fifo_if.empty;	■
Whenever internal counter is = FIFO_DEPTH, the full signal is = 1	**@(posedge fifo_if.clk)** count_dbg == FIFO_DEPTH **|->** fifo_if.full;	☐
Whenever internal counter is not = FIFO_DEPTH, the full signal is = 0.	**@(posedge fifo_if.clk)** count_dbg **!=** FIFO_DEPTH **|->** **!**fifo_if.full;	☐
When the almostfull is asserted and a write operation is required without a read operation then full signal will be = 1	**@(posedge fifo_if.clk)** fifo_if.almostfull **&&** fifo_if.wr_en **&&** **!**fifo_if.rd_en **|=>** fifo_if.full;	■
Whenever rd_en is asserted, full signal must = 0	**@(posedge fifo_if.clk)** fifo_if.rd_en **|=>** **!**fifo_if.full;	■
Whenever internal counter = FIFO_DEPTH – 1, the almostfull signal is asserted	**@(posedge fifo_if.clk)** count_dbg == FIFO_DEPTH - 1 **|->** fifo_if.almostfull;	☐
When internal counter != FIFO_DEPTH-1, the almostfull signal is deasserted	**@(posedge fifo_if.clk)** count_dbg **!=** FIFO_DEPTH - 1 **|->** **!**fifo_if.almostfull;	☐
When the FIFO is full and rd_en is asserted while wr_en is deasserted, the almostfull signal will assert	**@(posedge fifo_if.clk)** fifo_if.full **&&** fifo_if.rd_en **|=>** fifo_if.almostfull;	■
Whenever internal counter = 1, the almostempty signal is asserted	**@(posedge fifo_if.clk)** count_dbg == 1 **|->** fifo_if.almostempty;	☐
When internal counter != 1, the almostempty signal is deasserted	**@(posedge fifo_if.clk)** count_dbg **!=** 1 **|->** **!**fifo_if.almostempty;	☐
When the FIFO is empty and wr_en is asserted while rd_en is deasserted, the almostempty signal will assert	**@(posedge fifo_if.clk)** fifo_if.empty **&&** fifo_if.wr_en **|=>** fifo_if.almostempty;	■
When wr_ptr reaches maximum value, check wrapping around in case wr_en is asserted while the internal counter is not = FIFO_DEPTH	**@(posedge fifo_if.clk)** wr_ptr_dbg == 7 **&&** fifo_if.wr_en **&&** count_dbg < FIFO_DEPTH **|=>** wr_ptr_dbg == 0;	☐
When rd_ptr reaches maximum value, check wrapping around in case rd_en is asserted while the internal counter is not = 0	**@(posedge fifo_if.clk)** rd_ptr_dbg == 7 **&&** fifo_if.rd_en **&&** count_dbg > 0 **|=>** rd_ptr_dbg == 0;	☐